

주제: NLP 데이터 파이프라인 성능 및 구조 최적화

1. 연구 코드 소개

- 본 코드는 자연어 처리(NLP) 실험 및 데이터 전처리 파이프라인의 효율성을 검증하기 위한 기초 연구 코드임
- 약 50만 행의 텍스트 데이터셋을 기반으로 파일 입출력을 수행하고, 어휘집(Vocabulary)을 구축한 뒤, 각 단어의 가중치 상태를 업데이트하는 일련의 과정을 포함하고 있음
- 대규모 언어 모델이나 딥러닝 아키텍처를 본격적으로 도입하기 전, 텍스트 데이터를 메모리에 효율적으로 적재하고 배치(Batch) 단위로 분할하여 공급하는 방식을 결정하는 초기 핵심 파이프라인이기에 최적화 대상으로 선정함

2. 기존 코드의 문제점 분석

- 제공된 초기 baseline 코드를 분석하고 실행 환경에서 프로파일링을 수행한 결과, 다음과 같은 치명적인 성능 병목과 구조적 결함이 진단됨
- 메모리 자원 고갈 위험(Eager Loading): DataHandler 클래스에서 `f.readlines()`를 호출하여 대용량 텍스트 데이터를 한 번에 메모리에 적재함. 이는 데이터 스케일이 수십 GB 이상으로 확장될 경우 시스템 OOM(Out Of Memory) 에러를 유발하는 주원인임
- $O(N^2)$ 의 비효율적인 단어 카운팅: 고유 단어 목록을 생성하기 위해 전체 단어 리스트를 순회하며 `.count()`를 반복 호출함. 데이터 크기가 증가함에 따라 어휘집 구축 시간이 기하급수적으로 늘어나는 알고리즘적 한계를 보임
- 단일책임원칙(SRP) 위반 및 강한 결합도: 하이퍼파라미터 설정값들이 NLPModel 내부에 하드코딩되어 있으며, Trainer 클래스가 모델의 내부 가중치 디렉터리에 직접 접근하여 상태를 변조하는 등 객체지향 설계의 캡슐화 원칙을 심각하게 위반함
- 횡단 관심사의 혼재: 실행 시간 측정을 위한 `time.time()` 코드가 핵심 학습 로직 및 전처리 로직 사이에 사방으로 흩어져 있어 가독성과 재사용성을 저해함

3. 적용한 수업 개념

- 상기 문제점들을 해결하기 위해 파이썬 고급 프로그래밍 과정에서 학습한 4가지 핵심 개념을 유기적으로 적용함
- 자료구조 및 복잡도 개선(collections.Counter): 전체 리스트를 반복 순회하던 복잡도를 해시 테이블 기반의 단일 패스 스캔 연산으로 변경하여 시간복잡도를 $O(N)$ 으로 축소함
- 제너레이터(Generator)와 지연 평가(Lazy Evaluation): `yield` 키워드를 활용한 스트리밍 파이프라인을 구축하여, 대용량 파일을 한 번에 메모리에 올리지 않고 배치 사이즈만큼 동적으로 읽어 가며 소비하도록 설계함
- 클래스 기반 구조 개선 및 의존성 주입: `dataclass`를 활용해 하이퍼파라미터 설정을 불변 객체(RunConfig)로 격리하였고, 모델 내부의 가중치 데이터는 `@property` 및 전용 인터페이스 메서드를 통해서만 수정되도록 캡슐화 원칙을 적용함
- 데코레이터(Decorator) 패턴: 부가적인 시간 측정 로직을 독립된 모듈(`decorators.py`)로 분리하고 `functools.wraps`를 적용하여 원본 함수의 메타데이터를 보존하는 가독성 높은

구조를 완성함

4. 개선 과정

- 기존에 스파게티 형태로 얹혀 있던 6개의 파일을 엄격한 역할 분담에 따라 총 8개의 모듈 구조로 재설계함. 특별히 설정(Config)과 데코레이터 정책을 완전히 별도의 독립 파일로 격리함
- 가장 중요하게 고려한 지점은 DataHandler와 Trainer 간의 데이터 참조 의존성을 끊어내는 것이었음. 기존에는 학습 루프가 핸들러의 내부 리스트를 직접 호출했으나, 리팩터링 후에는 오직 batch_generator()가 반환하는 이터레이터 파이프라인만을 소비하도록 강제함
- 초기에 파일 수가 늘어남에 따라 구조적 오버헤드가 우려되었으나, 결과적으로 각 모듈이 단 하나의 책임만 지게 되어 디버깅 및 실험 조건 변경이 훨씬 용이해짐

5. 최적화 결과

- 합성 데이터셋(50만 줄, 약 35MB)을 기준으로 3회 반복 측정한 정량적 평균 성능 비교 지표는 다음과 같음
- 측정 환경: Python 3.12 / OS 환경 내 memory_profiler 패키지 활용 / 16GB RAM

평가 지표	최적화 전 (Before)	최적화 후 (After)	성능 개선율
전체 학습 스크립트 시간	평균 1.73초	평균 1.34초	약 22.3% 효율 향상
피크 메모리 사용량	약 380 MB	약 42 MB	약 89% 자원 절감
구조 설계 및 결합도	미준수 (강한 결합)	SRP 준수 (독립 모듈화)	확장성 및 유지보수성 극대화

6. 결과 해석 및 한계

- 정량적 지표가 증명하듯 Counter 자료구조의 도입과 제너레이터 기반의 지연 평가 처리가 전체 실행 시간을 약 21% 단축시켰으며, 피크 메모리 사용량을 10분의 1 수준으로 억제하는 데 결정적인 역할을 함. 자원이 제한된 시스템 환경에서도 대용량 데이터셋을 안정적으로 다룰 수 있는 기반을 마련함
- 그러나 이번 구조 개선을 진행하며 발생한 명확한 한계점(Trade-off)도 확인됨. 제너레이터를 도입하여 파일 포인터를 기반으로 스트리밍 처리를 수행하다 보니, 전체 데이터셋을 메모리에 올려두고 무작위로 켜는 무작위 셔플(Shuffle) 연산을 가동할 수 없게 됨
- 이를 해결하기 위해서는 학습 프로세스 진입 전, 물리적인 데이터셋 파일을 사전에 chunk 단위로 분할하여 외부에서 셔플링하는 별도의 전처리 단계를 추가해야 하는 제약이 생김
- 또한 데코레이터 적용으로 인해 미세한 함수 호출 오버헤드가 발생하지만, 이는 모듈의 가독성과 코드 유지보수성 향상이라는 이점이 주는 가치에 비해 충분히 수용할 수 있는 비용으로 판단됨